

Synopsis: AI Academic Study Copilot & Coach

1. INTRODUCTION

Purpose

This document outlines the complete software requirements for the AI Academic Study Copilot. The application is a full stack, intelligent study assistant designed to manage learning schedules, quantify study effort, generate interactive materials, and provide context aware AI tutoring.

Scope

The system provides a centralized dashboard for tracking quantitative study metrics, a dedicated split pane learning environment with an integrated Pomodoro timer, an AI chat interface grounded in user specific notes, a syllabus parsing engine, and a gamified activity tracker. The scope encompasses the frontend Single Page Application (SPA), the backend API architecture, the relational database structure, and the AI/RAG (Retrieval Augmented Generation) pipeline necessary to power the dynamic features.

2. OVERALL DESCRIPTION

Product Perspective

The platform operates on a modern decoupled architecture. The frontend is a responsive React SPA utilizing advanced UI paradigms (resizable panes, drag and drop, interactive charts). The backend operates via a high performance Python/FastAPI server, managing state persistence in a PostgreSQL database, while a vector database manages the semantic retrieval required for the AI coach's context awareness.

Target Audience

Students: Managing structured university courses, processing dense syllabi, and preparing for examinations. Self Learners: Tracking independent study hours, maintaining learning streaks, and utilizing on demand AI tutoring for complex subjects.

3. FUNCTIONAL REQUIREMENTS

Authentication & User Management

System Access: Users must be able to securely register and log in using email/password or OAuth providers (e.g., Google). Profile State: The backend must maintain user profiles, linking them securely to their uploaded syllabi, generated notes, and historical study data via unique session tokens (JWT).

Dashboard & Quantitative Analytics

Quick Start Protocol: The dashboard must feature a primary call to action that fetches the user's last active session ID from the backend and instantly loads the corresponding module. **Real Time Metrics:** Display "Study Time Today," "Current Streak," and "Topics Covered." **Study Hours Progress Chart:** A graphical Bar Chart tracking exact hours studied. **Backend Logic:** The system records time exclusively based on quantifiable effort (completed Pomodoro intervals or logged active hours), explicitly ignoring subjective concept mastery metrics. Data is aggregated via PostgreSQL timeseries queries.

Dynamic Study Session Environment

Resizable Split Layout: The frontend must utilize a drag to resize paradigm to divide the workspace between study materials (left) and the AI coach (right). **Integrated Pomodoro Timer:** State management for Work (25 min), Short Break (5 min), and Long Break (15 min) modes. Syncing requires completed work intervals to send a payload to the backend to update the user's daily study hours and trigger streak calculations. **Left Panel (Study Materials):** Smart Notes renders markdown formatted course notes generated by the backend AI processing pipeline. Interactive Quizzes provide dynamic multiple choice forms rendered from structured JSON payloads. **Right Panel (AI Study Coach):** A persistent, real time chat interface. Context Awareness ensures every message sent to the AI must bundle metadata about the user's active tab, current topic, and recent notes. The backend RAG pipeline injects this specific context into the LLM prompt to ensure highly relevant, localized tutoring without hallucination.

Syllabus Management & Ingestion Pipeline

Drag and Drop Upload: The UI must accept PDF and text files via a dedicated dropzone. **Processing Engine:** The backend extracts raw text from the uploaded document. An AI orchestration agent structures the raw text into a hierarchical JSON format (Units, Topics, Subtopics). The structured outline is saved to the database and populates the user's course navigation menu.

Contribution & Gamification Engine

Activity Heat Map: A 365 day contribution matrix (similar to GitHub) visualizing daily study intensity. The backend API must return an array of date value pairs representing study duration. **Streak Calculation:** A background service or database trigger evaluates consecutive days where `Study_Hours > 0` to update the user's "Current Streak" and "Personal Best" variables.

4. NON FUNCTIONAL REQUIREMENTS

UI/UX & Theming

Theme Support: Full Light/Dark mode toggling, persisting the preference in local storage and applying CSS variables globally. **Responsive Degradation:** The split pane layout must collapse gracefully into a stacked single column view on mobile devices, and non essential UI elements (like Long Break buttons) should be hidden on narrow viewports. **Fluid Interactions:** Panel resizing, chart loading, and state transitions must utilize smooth physics based animations (maintaining 60 FPS) without obstructing the core reading experience.

System Performance & Scalability

Timer Precision: The Pomodoro timer must rely on system clock timestamps rather than pure interval ticks to prevent drift and UI freezing during heavy processing. API Latency: Standard CRUD operations and analytics retrieval should resolve in under 200ms. Streaming Responses: The AI Coach chat interface must utilize Server Sent Events (SSE) or WebSockets to stream text responses chunk by chunk, ensuring immediate visual feedback.

Security & Data Integrity

Payload Validation: All incoming API requests must be strictly validated against predefined schemas to prevent injection attacks. Data Isolation: Database queries must enforce tenant isolation, ensuring users can only access and query their own uploaded syllabi and generated notes.

5. TECHNOLOGY STACK

Frontend Architecture

Core Framework: React 18+
Routing: React Router (Data Mode via RouterProvider)
Styling: Tailwind CSS v4
Icons: Lucide React
Complex UI: react resizable panels (for split study environment)

Backend & Data Architecture

Core Framework: Python with FastAPI (for high performance, asynchronous endpoints)
Primary Database: PostgreSQL (managed via SQLAlchemy ORM for relational data, users, and session states)
Vector Database: ChromaDB (for storing and querying text embeddings of uploaded syllabi and generated notes)
AI Orchestration: LangGraph / LangChain (for structuring the RAG pipeline and managing the state of the AI Study Coach)

COMMENTS

SIGNATURE

Authorized Signature / Date